GAME PROGRAMMING COURSE — SAVE DATA

データのセーブ

SAVE

PlayerPrefs と JSON、そしてセーブスロット

対象：クラスまで既習の選抜クラス

今日のゴール

クリア状況とベストタイムを、ゲームを閉じても残す

① PlayerPrefs

いちばん手軽な保存方法。
キーと値で1つずつ保存する。

② JSON

データを**クラスにまとめて**、
ファイルに丸ごと保存する。

③ セーブスロット

JSON ファイルを**スロットの数だけ**用意する。
今日は紹介まで。

POINT 作り替えるのは **SaveDataManager** だけ。使う側（SceneData・StageTimer）は**一度直したら、あとは変わらない。**

PART 1

セーブの基本

1

シーンをまたいで残る ≠ ゲームを閉じても残る

なぜセーブが必要？

PersistentSingleton でも、閉じたら消える

ゲーム起動
SaveDataManager が作られる

→

シーンを移動
DontDestroyOnLoad で残る

→

ゲームを閉じる
メモリごと消える

メモリ（変数）

- 変数の値は**メモリ**の上にある
- ゲームを閉じると、メモリは**全部片付けられる**

セーブ（ファイル）

- **ファイル**などに書き出しておけば、閉じても残る
- 次に起動したとき**読み込んで**変数に戻す

決めること

何を・いつ セーブ／ロードするか

	今日のゲームでは
何を保存する？	① ステージ（Tutorial / FirstIngame / SecondIngame）のクリア状況 ② シーンごとのベストタイム
いつセーブする？	ステージをクリアした瞬間／ベストタイムを更新した瞬間
いつロードする？	ゲーム起動時（SaveDataManager の Awake）
どこから呼ぶ？	SceneData → SetClear()　／　StageTimer → TryUpdateBestTime()

POINT　ほかによくあるタイミング：**チェックポイント**、**メニューのセーブボタン**、**オートセーブ**（一定時間ごと）、**ゲーム終了時**。

2つの方法

PlayerPrefs と JSON

	PlayerPrefs	JSON ＋ ファイル
保存のしかた	キーと値を1つずつ	クラスを丸ごと文字列にして保存
使える型	int / float / string だけ	配列・List・クラスの入れ子もOK
保存先	Unity が決めた場所（OS ごとに違う）	自分で決めたファイル
向いているもの	音量・画面設定など少しの値	セーブデータ本体・たくさんの値
スロット	キー名を工夫すればできるが大変	ファイルを分けるだけ

PART 2

PlayerPrefs でセーブ

2

キーと値で、1つずつ保存する

PlayerPrefs の使い方

名前（キー）を付けて保存・読み込み

C# PlayerPrefs の基本

```
PlayerPrefs.SetInt("Coin", 120);          // 保存（int）
PlayerPrefs.SetFloat("Volume", 0.8f);     // 保存（float）
PlayerPrefs.SetString("Name", "Hero");    // 保存（string）
PlayerPrefs.Save();                       // 確実にディスクへ書き込む

int coin = PlayerPrefs.GetInt("Coin", 0);  // 読み込み（なければ 0）
bool has = PlayerPrefs.HasKey("Coin");     // 保存されている？
PlayerPrefs.DeleteKey("Coin");             // 消す
```

POINT PlayerPrefs は **Dictionary と同じ「キー → 値」**。ちがうのは、ゲームを閉じても**残る**こと。

SaveDataManager.cs（PlayerPrefs 版）

C# Assets/Scripts/SaveDataManager.cs

```cs
using System.Collections.Generic;
using UnityEngine;

// クリア状況とベストタイムを持ち、PlayerPrefs にセーブする
public class SaveDataManager : PersistentSingleton<SaveDataManager>
{
    // 保存するときの名前（キー）の頭に付ける文字
    private const string CLEAR_KEY_PREFIX = "IsClear_";
    private const string BEST_TIME_KEY_PREFIX = "BestTime_";

    private bool[] isClears = new bool[SceneData.INGAME_COUNT];
    private Dictionary<string, float> bestTimes = new Dictionary<string, float>();

    [RuntimeInitializeOnLoadMethod]
    static void Init()
    {
        SaveDataManager pref = Resources.Load<SaveDataManager>("Singletons/SaveDataManager");
        if (pref != null) Instantiate(pref);
    }
```

SaveDataManager.cs（PlayerPrefs 版）

C# Assets/Scripts/SaveDataManager.cs

```
21      protected override void Awake()
22      {
23          base.Awake();
24          Load();
25      }
26
27      // ===== クリア =====
28      public bool IsCleared(int index)
29      {
30          return isClears[index];
31      }
32
33      public void SetClear(int index)
34      {
35          isClears[index] = true;
36          Save();
37      }
```

SaveDataManager.cs（PlayerPrefs 版）

C# Assets/Scripts/SaveDataManager.cs

```
39      // ===== ベストタイム =====
40      /// <summary>
41      /// 引数の値がベストタイムなら保存を更新する
42      /// </summary>
43      /// <param name="sceneName">シーン名</param>
44      /// <param name="time">現在記録</param>
45      /// <returns>更新したら true</returns>
46      public bool TryUpdateBestTime(string sceneName, float time)
47      {
48          bool isGet = bestTimes.TryGetValue(sceneName, out float best);
49          if (isGet && best <= time)
50          {
51              return false;
52          }
53
54          bestTimes[sceneName] = time;
55          Save();
56          return true;
57      }
```

SaveDataManager.cs（PlayerPrefs 版）

C# Assets/Scripts/SaveDataManager.cs

```
59      public bool HasBestTime(string sceneName)
60      {
61          return bestTimes.ContainsKey(sceneName);
62      }
63
64      public float GetBestTime(string sceneName)
65      {
66          return bestTimes[sceneName];
67      }
```

SaveDataManager.cs（PlayerPrefs 版）

C# Assets/Scripts/SaveDataManager.cs

```
69      // ===== セーブ・ロード =====
70      public void Save()
71      {
72          // クリア状況：bool は保存できないので 1 / 0 にする
73          for (int i = 0; i < isClears.Length; i++)
74          {
75              PlayerPrefs.SetInt(CLEAR_KEY_PREFIX + i, isClears[i] ? 1 : 0);
76          }
77
78          // ベストタイム：シーン名をキーに入れて1つずつ保存
79          foreach (KeyValuePair<string, float> pair in bestTimes)
80          {
81              PlayerPrefs.SetFloat(BEST_TIME_KEY_PREFIX + pair.Key, pair.Value);
82          }
83
84          PlayerPrefs.Save();
85      }
```

SaveDataManager.cs（PlayerPrefs 版）

C# Assets/Scripts/SaveDataManager.cs

```
87      public void Load()
88      {
89          for (int i = 0; i < isClears.Length; i++)
90          {
91              isClears[i] = PlayerPrefs.GetInt(CLEAR_KEY_PREFIX + i, 0) == 1;
92          }
93
94          // PlayerPrefs は「保存したキーの一覧」を取れないので、全シーン名を調べる
95          bestTimes.Clear();
96          for (int i = 0; i < (int)SceneName.Max; i++)
97          {
98              string sceneName = SceneData.GetSceneName((SceneName)i);
99              string key = BEST_TIME_KEY_PREFIX + sceneName;
100             if (PlayerPrefs.HasKey(key))
101             {
102                 bestTimes[sceneName] = PlayerPrefs.GetFloat(key);
103             }
104         }
105     }
106 }
```

キーの頭に付ける文字を定数にする

C# Assets/Scripts/SaveDataManager.cs

```
7       // 保存するときの名前（キー）の頭に付ける文字
8       private const string CLEAR_KEY_PREFIX = "IsClear_";
9       private const string BEST_TIME_KEY_PREFIX = "BestTime_";
10
11      private bool[] isClears = new bool[SceneData.INGAME_COUNT];
12      private Dictionary<string, float> bestTimes = new Dictionary<string, float>();
```

- キーは**ただの文字列** → 打ち間違えても**エラーにならない**
- **IsClear_0**、**BestTime_Tutorial_0** のように後ろをつなげて使う
- GameDataHolder の **isClears** をこっちへ引っ越し

起動したら読み込み、クリアしたらすぐ保存

C# Assets/Scripts/SaveDataManager.cs

```
21      protected override void Awake()
22      {
23          base.Awake();
24          Load();
25      }
26
27      // ===== クリア =====
28      public bool IsCleared(int index)
29      {
30          return isClears[index];
31      }
32
33      public void SetClear(int index)
34      {
35          isClears[index] = true;
36          Save();
37      }
```

ポイント

- **Awake** で Load()：起動時に1回
- **SetClear**：記録したら**その場で Save()**
- 外からは IsCleared / SetClear だけ使う

ベストタイムは「更新したとき」だけ保存

C# Assets/Scripts/SaveDataManager.cs

```
46      public bool TryUpdateBestTime(string sceneName, float time)
47      {
48          bool isGet = bestTimes.TryGetValue(sceneName, out float best);
49          if (isGet && best <= time)
50          {
51              return false;
52          }
53
54          bestTimes[sceneName] = time;
55          Save();
56          return true;
57      }
```

ポイント

- 前回のコードに **Save(); を1行**足すだけ
- 遅かったとき（return false）は**保存しない**
- StageTimer 側は**何も変えない**

Save ① — クリア状況は 1 / 0 で保存

C# Assets/Scripts/SaveDataManager.cs

```
70      public void Save()
71      {
72          // クリア状況：bool は保存できないので 1 / 0 にする
73          for (int i = 0; i < isClears.Length; i++)
74          {
75              PlayerPrefs.SetInt(CLEAR_KEY_PREFIX + i, isClears[i] ? 1 : 0);
76          }
```

ポイント

- キーは **IsClear_0〜2**
- bool は **? 1 : 0** で int にする
- true なら 1、false なら 0

Save ② — ベストタイムはシーン名をキーに

C# Assets/Scripts/SaveDataManager.cs

```
78          // ベストタイム：シーン名をキーに入れて1つずつ保存
79          foreach (KeyValuePair<string, float> pair in bestTimes)
80          {
81              PlayerPrefs.SetFloat(BEST_TIME_KEY_PREFIX + pair.Key, pair.Value);
82          }
83
84          PlayerPrefs.Save();
85      }
```

- Dictionary は **foreach** で1件ずつ
- **pair.Key** ＝シーン名、**pair.Value** ＝タイム
- キーは **BestTime_Tutorial_0** など
- 最後に **PlayerPrefs.Save()**

Load — 保存したキーを全部探して戻す

C# Assets/Scripts/SaveDataManager.cs

```
87      public void Load()
88      {
89          for (int i = 0; i < isClears.Length; i++)
90          {
91              isClears[i] = PlayerPrefs.GetInt(CLEAR_KEY_PREFIX + i, 0) == 1;
92          }
93
94          // PlayerPrefs は「保存したキーの一覧」を取れないので、全シーン名を調べる
95          bestTimes.Clear();
96          for (int i = 0; i < (int)SceneName.Max; i++)
97          {
98              string sceneName = SceneData.GetSceneName((SceneName)i);
99              string key = BEST_TIME_KEY_PREFIX + sceneName;
100             if (PlayerPrefs.HasKey(key))
101             {
102                 bestTimes[sceneName] = PlayerPrefs.GetFloat(key);
103             }
104         }
105     }
```

ポイント

- クリア：**GetInt(キー, 0) == 1**
- PlayerPrefs は**保存したキーの一覧が取れない**
- → **全シーン名**を回して **HasKey** で確認

使う側を直す

SceneData から SetClear を呼ぶ

C# Assets/Scripts/SceneDatas.cs

```
56          if (nextNumber == 3)
57          {
58              switch (sceneNameType)
59              {
60                  case "Tutorial": SaveDataManager.Instance.SetClear(0); break;
61                  case "FirstIngame": SaveDataManager.Instance.SetClear(1); break;
62                  case "SecondIngame": SaveDataManager.Instance.SetClear(2); break;
63              }
64              return GetSceneName(SceneName.Title);
```

- GameDataHolder.Instance.isClears[0] = true; を
- **SaveDataManager.Instance.SetClear(0)** に（3行）
- このあと JSON 版にしても、**ここはもう変えない**

ここまでで確かめること

1. Tutorial を _2 までクリアする
2. **再生を止めて**、もう一度再生する
3. クリア状況（IsCleared(0)）と**ベストタイム**が残っている
4. 消したいときは **Edit › Clear All PlayerPrefs**

PlayerPrefs の向き・不向き

設定には便利、セーブデータ本体には不向き

向いている

- 音量・言語・画面モードなどの**設定**
- 「最後に選んだスロット」のような**小さな値**
- 数行で書ける

向いていない

- 値が増えるたびに**キーと Set / Get が増える**
- **キーの一覧が取れない**（Load で全シーンを探した）
- 配列・Dictionary は**そのまま保存できない**

POINT クリア状況・ベストタイム・所持アイテム…と増えていくセーブデータは、**クラスにまとめて JSON** にする。

PART 3

JSON でセーブ

3

データをクラスにまとめて、ファイルに丸ごと保存する

JSON とは

データを文字で書き表すルール

C# save.json（実際に保存されるファイル）

```
{
    "isClears": [
        true,
        false,
        false
    ],
    "bestTimes": [
        {
            "sceneName": "Tutorial_0",
            "time": 12.5
        },
        {
            "sceneName": "Tutorial_1",
            "time": 20.25
        }
    ]
}
```

読み方

- **{ }**：ひとまとまり（クラスのイメージ）
- **"名前": 値**：変数名と値
- **[ ]**：配列・List
- ただのテキストなので、**メモ帳で開ける**

POINT Unity の **JsonUtility** を使うと、クラス ⇔ JSON の文字列 を1行で変換できる。

JsonUtility の注意

そのまま保存できるもの・できないもの

型	JsonUtility で	どうする？
int / float / bool / string	できる	そのまま
配列・List<T>	できる	そのまま
[Serializable] を付けたクラス	できる（入れ子もOK）	そのまま
Dictionary	できない	キーと値のクラスを作って List にする
private の変数	保存されない	public か [SerializeField]
プロパティ（get; set;）	保存されない	変数にする

注意 bestTimes は **Dictionary** … そのままでは保存できない！

Dictionary を List に詰め替える

1件を「シーン名とタイムを持つクラス」にする

Dictionary<string, float>（ゲーム中に使う）

```
"Tutorial_0" → 12.5
"Tutorial_1" → 20.25
```

Save

List<BestTimeRecord>（JSON にする）

```
{ sceneName:"Tutorial_0", time:12.5 }
{ sceneName:"Tutorial_1", time:20.25 }
```

- **Save**：Dictionary → List（foreach で1件ずつ BestTimeRecord を作って Add）
- **Load**：List → Dictionary（foreach で1件ずつ戻す）
- ゲーム中は今までどおり **Dictionary** で使える（TryUpdateBestTime はそのまま）

SaveDataManager.cs（JSON 版）

C# Assets/Scripts/SaveDataManager.cs

```cs
using System;
using System.Collections.Generic;
using System.IO;
using UnityEngine;

// セーブする中身をまとめたクラス（これを丸ごと JSON にする）
[Serializable]
public class SaveData
{
    public bool[] isClears = new bool[SceneData.INGAME_COUNT];
    public List<BestTimeRecord> bestTimes = new List<BestTimeRecord>();
}

// ベストタイム1件分（Dictionary は JSON にできないので、キーと値をクラスにする）
[Serializable]
public class BestTimeRecord
{
    public string sceneName;
    public float time;
}
```

SaveDataManager.cs（JSON 版）

C# Assets/Scripts/SaveDataManager.cs

```
22  // クリア状況とベストタイムを持ち、JSON ファイルにセーブする
23  public class SaveDataManager : PersistentSingleton<SaveDataManager>
24  {
25      private const string SAVE_FILE_NAME = "save.json";
26
27      private bool[] isClears = new bool[SceneData.INGAME_COUNT];
28      private Dictionary<string, float> bestTimes = new Dictionary<string, float>();
29
30      // 保存先：書き込んでよいフォルダ + ファイル名
31      private string SavePath => Path.Combine(Application.persistentDataPath, SAVE_FILE_NAME);
32
33      [RuntimeInitializeOnLoadMethod]
34      static void Init()
35      {
36          SaveDataManager pref = Resources.Load<SaveDataManager>("Singletons/SaveDataManager");
37          if (pref != null) Instantiate(pref);
38      }
```

SaveDataManager.cs（JSON 版）

C# Assets/Scripts/SaveDataManager.cs

```
40      protected override void Awake()
41      {
42          base.Awake();
43          Load();
44      }
45
46      // ===== クリア =====
47      public bool IsCleared(int index)
48      {
49          return isClears[index];
50      }
51
52      public void SetClear(int index)
53      {
54          isClears[index] = true;
55          Save();
56      }
```

SaveDataManager.cs（JSON 版）

C# Assets/Scripts/SaveDataManager.cs

```
58      // ===== ベストタイム =====
59      /// <summary>
60      /// 引数の値がベストタイムなら保存を更新する
61      /// </summary>
62      /// <param name="sceneName">シーン名</param>
63      /// <param name="time">現在記録</param>
64      /// <returns>更新したら true</returns>
65      public bool TryUpdateBestTime(string sceneName, float time)
66      {
67          bool isGet = bestTimes.TryGetValue(sceneName, out float best);
68          if (isGet && best <= time)
69          {
70              return false;
71          }
72
73          bestTimes[sceneName] = time;
74          Save();
75          return true;
76      }
```

SaveDataManager.cs（JSON 版）

C# Assets/Scripts/SaveDataManager.cs

```
78      public bool HasBestTime(string sceneName)
79      {
80          return bestTimes.ContainsKey(sceneName);
81      }
82
83      public float GetBestTime(string sceneName)
84      {
85          return bestTimes[sceneName];
86      }
```

SaveDataManager.cs（JSON 版）

C# Assets/Scripts/SaveDataManager.cs

```
88      // ===== セーブ・ロード =====
89      public void Save()
90      {
91          SaveData data = new SaveData();
92          data.isClears = isClears;
93
94          // Dictionary を List に詰め替える
95          foreach (KeyValuePair<string, float> pair in bestTimes)
96          {
97              BestTimeRecord record = new BestTimeRecord();
98              record.sceneName = pair.Key;
99              record.time = pair.Value;
100             data.bestTimes.Add(record);
101         }
102
103         string json = JsonUtility.ToJson(data, true);
104         File.WriteAllText(SavePath, json);
105         Debug.Log($"セーブしました：{SavePath}");
106     }
```

SaveDataManager.cs（JSON 版）

C# Assets/Scripts/SaveDataManager.cs

```
108     public void Load()
109     {
110         // ファイルがない＝初めて遊ぶ人
111         if (!File.Exists(SavePath)) return;
112 
113         string json = File.ReadAllText(SavePath);
114         SaveData data = JsonUtility.FromJson<SaveData>(json);
115 
116         isClears = data.isClears;
117 
118         // List を Dictionary に戻す
119         bestTimes.Clear();
120         foreach (BestTimeRecord record in data.bestTimes)
121         {
122             bestTimes[record.sceneName] = record.time;
123         }
124     }
125 }
```

JSON 版　解説 ①

セーブ用のクラスを、同じファイルの先頭に書く

C# Assets/Scripts/SaveDataManager.cs

```
6  // セーブする中身をまとめたクラス（これを丸ごと JSON にする）
7  [Serializable]
8  public class SaveData
9  {
10     public bool[] isClears = new bool[SceneData.INGAME_COUNT];
11     public List<BestTimeRecord> bestTimes = new List<BestTimeRecord>();
12 }
13
14 // ベストタイム1件分（Dictionary は JSON にできないので、キーと値をクラスにする）
15 [Serializable]
16 public class BestTimeRecord
17 {
18     public string sceneName;
19     public float time;
20 }
```

ポイント

- **[Serializable]** を付ける（using System;）
- 保存する変数は **public**
- MonoBehaviour は**継承しない**（ただのデータ入れ）
- **BestTimeRecord** = Dictionary の1件分

JSON 版　解説 ②

保存先を決める

C# Assets/Scripts/SaveDataManager.cs

```
25      private const string SAVE_FILE_NAME = "save.json";
26
27      private bool[] isClears = new bool[SceneData.INGAME_COUNT];
28      private Dictionary<string, float> bestTimes = new Dictionary<string, float>();
29
30      // 保存先：書き込んでよいフォルダ ＋ ファイル名
31      private string SavePath => Path.Combine(Application.persistentDataPath, SAVE_FILE_NAME);
```

- キーの定数は**いらなくなる**
- **persistentDataPath**：書き込んでよいフォルダ
- **Path.Combine**：フォルダとファイル名をつなぐ

JSON 版　解説 ③

Save — まず SaveData に詰める

C# Assets/Scripts/SaveDataManager.cs

```
89      public void Save()
90      {
91          SaveData data = new SaveData();
92          data.isClears = isClears;
93
94          // Dictionary を List に詰め替える
95          foreach (KeyValuePair<string, float> pair in bestTimes)
96          {
97              BestTimeRecord record = new BestTimeRecord();
98              record.sceneName = pair.Key;
99              record.time = pair.Value;
100             data.bestTimes.Add(record);
101         }
```

ポイント

- SaveData を **new** して中身を入れる
- isClears は**配列なのでそのまま**
- bestTimes は **1件ずつ BestTimeRecord** にして Add

JSON 版　解説 ④

Save — クラス → 文字列 → ファイル

C# Assets/Scripts/SaveDataManager.cs

```
103         string json = JsonUtility.ToJson(data, true);
104         File.WriteAllText(SavePath, json);
105         Debug.Log($"セーブしました：{SavePath}");
106     }
```

ポイント

- **JsonUtility.ToJson**：クラスを JSON の文字列に
- **File.WriteAllText**：文字列をファイルに書く（上書き）
- 保存先を Debug.Log で出しておくと探しやすい

JSON 版　解説 ⑤

Load — ファイル → 文字列 → クラス → 変数

C# Assets/Scripts/SaveDataManager.cs

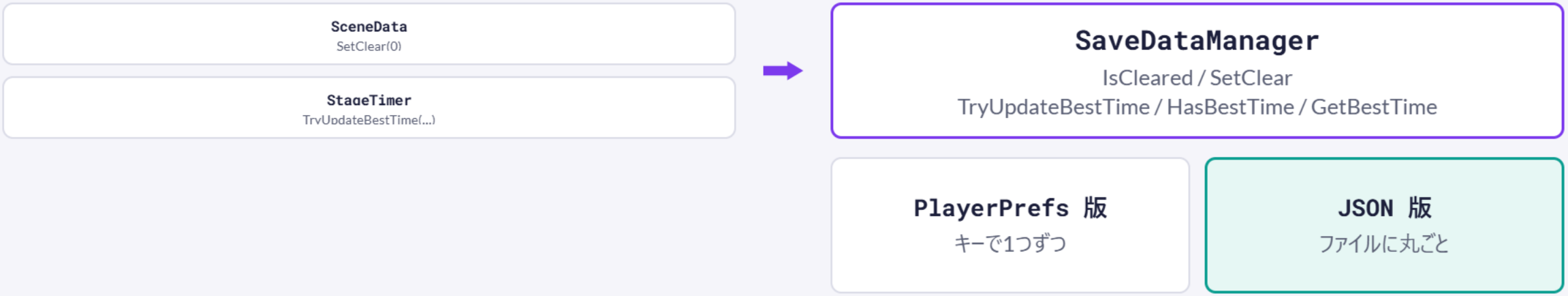
```
108     public void Load()
109     {
110         // ファイルがない＝初めて遊ぶ人
111         if (!File.Exists(SavePath)) return;
112 
113         string json = File.ReadAllText(SavePath);
114         SaveData data = JsonUtility.FromJson<SaveData>(json);
115 
116         isClears = data.isClears;
117 
118         // List を Dictionary に戻す
119         bestTimes.Clear();
120         foreach (BestTimeRecord record in data.bestTimes)
121         {
122             bestTimes[record.sceneName] = record.time;
123         }
124     }
```

ポイント

- ファイルがなければ**何もしない**
- **ReadAllText → FromJson<SaveData>**
- List を **Dictionary** に戻す

ポイント

中身を変えても、使う側は変わらない



POINT 外から見える**メソッドの名前と使い方を変えない**ので、保存のしかたを入れ替えても SceneData・StageTimer は**1行も直さなくてよい**（カプセル化）。

ここまでで確かめること

1. ステージをクリアすると、Console に**保存先のパス**が出る
2. そのフォルダを開くと **save.json** がある（メモ帳で開ける）
3. 再生し直しても**クリア状況・ベストタイムが残っている**
4. save.json を**消す**と、初めから遊べる

PART 3　演習

死んだ回数を記録しよう

JSON 版の SaveDataManager に、セーブするものを1つ足す

問題

死んだ回数を記録して、画面に出す

	仕様
今回の回数	今の挑戦（そのシーンをクリアするまで）で死んだ回数　→　セーブしない
最大の回数	そのシーンで、1回の挑戦でいちばん多く死んだ回数　→　JSON にセーブ
増える・戻る	死んだら +1（最大を超えたら最大も更新）／クリアしたら今回を 0 に
表示	ステージ画面に「ミス　今回 2 回／最大 5 回」のように出す

POINT　最大回数は**ベストタイムと同じく、シーンごと**に記録する。ゲームを閉じても残ること。

ヒント ①

何を、どこに持つ？

SaveDataManager

- 今回の回数：**int** 1つ（セーブしない）
- 最大の回数：**Dictionary<string, int>**（シーン名 → 回数）
- 外から読むなら**プロパティ**や **Get〜** メソッド

SaveData

- Dictionary は JSON にできない……
- → ベストタイムと**同じ形**のクラスを作って **List** に
- Save で詰め替え、Load で戻す

POINT **bestTimes でやったことを、そのままもう1回**やればよい。

ヒント ②

どこから呼ぶ？いつ表示する？



- シーン名は **SceneManager.GetActiveScene().name**
- 死ぬと**シーンが読み込み直される** → 表示は **Start** で1回やれば最新になる

PART 3　演習

解答例

SaveDataManager に足したところを順に見る

解答 ①

SaveData に List とクラスを足す

C# Assets/Scripts/SaveDataManager.cs

```
6  // セーブする中身をまとめたクラス（これを丸ごと JSON にする）
7  [Serializable]
8  public class SaveData
9  {
10     public bool[] isClears = new bool[SceneData.INGAME_COUNT];
11     public List<BestTimeRecord> bestTimes = new List<BestTimeRecord>();
12     public List<DeathCountRecord> maxDeathCounts = new List<DeathCountRecord>();
13 }
```

C# Assets/Scripts/SaveDataManager.cs

```
23 // 最大死亡回数1件分（ベストタイムと同じ形）
24 [Serializable]
25 public class DeathCountRecord
26 {
27     public string sceneName;
28     public int count;
29 }
```

ポイント

- **DeathCountRecord**：シーン名と回数
- BestTimeRecord と**同じ形**
- SaveData に **List** を1つ足す

解答 ②

SaveDataManager に変数を足す

C# Assets/Scripts/SaveDataManager.cs

```
36      private bool[] isClears = new bool[SceneData.INGAME_COUNT];
37      private Dictionary<string, float> bestTimes = new Dictionary<string, float>();
38      private Dictionary<string, int> maxDeathCounts = new Dictionary<string, int>();
39
40      // 今の挑戦で死んだ回数（セーブしない）
41      public int CurrentDeathCount { get; private set; }
```

- 最大：**Dictionary<string, int>**
- 今回：**{ get; private set; }**
- 外からは読めるが、**書き換えられない**

解答 ③

AddDeath — 増やして、最大を超えたら保存

C# Assets/Scripts/SaveDataManager.cs

```
101     // ===== 死亡回数 =====
102     // 死んだ回数を1増やし、最大回数を超えたら更新して保存する
103     public void AddDeath(string sceneName)
104     {
105         CurrentDeathCount++;
106 
107         if (CurrentDeathCount <= GetMaxDeathCount(sceneName)) return;
108 
109         maxDeathCounts[sceneName] = CurrentDeathCount;
110         Save();
111     }
```

ポイント

- まず **+1**
- 最大**以下**なら return（何もしない）
- 超えたら最大を更新して **Save()**

解答 ④

最大を読む・今回を 0 に戻す

C# Assets/Scripts/SaveDataManager.cs

```
113     // 最大死亡回数（まだ記録がなければ 0）
114     public int GetMaxDeathCount(string sceneName)
115     {
116         maxDeathCounts.TryGetValue(sceneName, out int max);
117         return max;
118     }
119
120     // クリアしたら、次の挑戦のために 0 に戻す
121     public void ResetCurrentDeathCount()
122     {
123         CurrentDeathCount = 0;
124     }
```

ポイント

- **TryGetValue**：なければ max は **0** のまま
- → 初めてのシーンでも**エラーにならない**
- **Reset**：クリアしたときに呼ぶ

解答 ⑤

Save で詰め替える

C# Assets/Scripts/SaveDataManager.cs

```
141         // 最大死亡回数も同じように詰め替える
142         foreach (KeyValuePair<string, int> pair in maxDeathCounts)
143         {
144             DeathCountRecord record = new DeathCountRecord();
145             record.sceneName = pair.Key;
146             record.count = pair.Value;
147             data.maxDeathCounts.Add(record);
148         }
```

ポイント

- ベストタイムの foreach を**まねして**もう1つ
- **pair.Value** は int

解答 ⑥

Load で Dictionary に戻す

C# Assets/Scripts/SaveDataManager.cs

```
172         maxDeathCounts.Clear();
173         foreach (DeathCountRecord record in data.maxDeathCounts)
174         {
175             maxDeathCounts[record.sceneName] = record.count;
176         }
```

ポイント

- まず **Clear()**
- List から1件ずつ **Dictionary** に戻す

全文

解答 ⑦　DeathCountLabel.cs（表示用）

C# Assets/Scripts/DeathCountLabel.cs

```
using TMPro;
using UnityEngine;
using UnityEngine.SceneManagement;

// 死んだ回数（今の挑戦・最大）を表示する
public class DeathCountLabel : MonoBehaviour
{
    [SerializeField] private TextMeshProUGUI label;

    // 死ぬとシーンが読み込み直されるので、Start で表示すれば毎回最新になる
    private void Start()
    {
        string sceneName = SceneManager.GetActiveScene().name;
        int current = SaveDataManager.Instance.CurrentDeathCount;
        int max = SaveDataManager.Instance.GetMaxDeathCount(sceneName);
        label.text = $"ミス　今回 {current} 回／最大 {max} 回";
    }
}
```

解答 ⑧

PlayerController — 死んだら AddDeath

C# Assets/Scripts/PlayerController.cs

```
208     public void GameOver()
209     {
210         if (IsGameEnd) return;
211         ChangeState(GameOverState);
212         SaveDataManager.Instance.AddDeath(SceneManager.GetActiveScene().name);  // 追加
213         Break();
214         InGameUi.Instance.GameOver();
215         TPSCameraController.Instance.ShakeCam();
216         StageTimer.Instance.StopTimer();
217     }
```

- **AddDeath** にシーン名を渡す
- ファイルの先頭に using **UnityEngine.SceneManagement**;

解答 ⑨

PlayerController — クリアしたら 0 に

C# Assets/Scripts/PlayerController.cs

```
219     public void GameClear()
220     {
221         if (IsGameEnd) return;
222         ChangeState(ClearState);
223         SaveDataManager.Instance.ResetCurrentDeathCount();  // 追加
224         InGameUi.Instance.GameClear();
225         StageTimer.Instance.RecordClear();
226     }
```

ポイント

- **ResetCurrentDeathCount** を呼ぶ
- 次のシーンの表示は「今回 0 回」になる

演習　動かしてみよう

ここまでで確かめること

1. わざと死ぬと「今回」が 1, 2, 3… と**増えていく**
2. 前の最大を超えると「最大」も**一緒に増える**
3. クリアすると、次のシーンで「今回」が **0** に戻る
4. 再生し直しても「最大」が**残っている**（save.json に maxDeathCounts がある）

PART 4

セーブスロット

4

JSON ファイルを、スロットの数だけ用意する

考え方

ファイル名にスロット番号を入れるだけ

save_0.json	save_1.json	save_2.json
SLOT 1：Tutorial クリア済み	SLOT 2：まだない	SLOT 3：全部クリア

- 今使っているスロット番号（**CurrentSlot**）を持つ
- セーブ・ロードは **save_{CurrentSlot}.json** に対して行う
- 「前回どのスロットで遊んだか」は小さな値なので **PlayerPrefs** に保存

POINT PlayerPrefs と JSON は**どちらか一方**ではない。**設定や小さな値は PlayerPrefs、データ本体は JSON** と使い分ける。

SaveDataManager.cs（スロット版）

C# Assets/Scripts/SaveDataManager.cs

```
using System;
using System.Collections.Generic;
using System.IO;
using UnityEngine;

// セーブする中身をまとめたクラス（これを丸ごと JSON にする）
[Serializable]
public class SaveData
{
    public bool[] isClears = new bool[SceneData.INGAME_COUNT];
    public List<BestTimeRecord> bestTimes = new List<BestTimeRecord>();
}

// ベストタイム1件分（Dictionary は JSON にできないので、キーと値をクラスにする）
[Serializable]
public class BestTimeRecord
{
    public string sceneName;
    public float time;
}
```

SaveDataManager.cs（スロット版）

C# Assets/Scripts/SaveDataManager.cs

```
22  // クリア状況とベストタイムを持ち、スロットごとの JSON ファイルにセーブする
23  public class SaveDataManager : PersistentSingleton<SaveDataManager>
24  {
25      public const int SLOT_COUNT = 3;
26      private const string LAST_SLOT_KEY = "LastSlot";
27
28      private bool[] isClears = new bool[SceneData.INGAME_COUNT];
29      private Dictionary<string, float> bestTimes = new Dictionary<string, float>();
30
31      // 今使っているスロット（0, 1, 2）
32      public int CurrentSlot { get; private set; }
33
34      [RuntimeInitializeOnLoadMethod]
35      static void Init()
36      {
37          SaveDataManager pref = Resources.Load<SaveDataManager>("Singletons/SaveDataManager");
38          if (pref != null) Instantiate(pref);
39      }
```

SaveDataManager.cs（スロット版）

C# Assets/Scripts/SaveDataManager.cs

```
41      protected override void Awake()
42      {
43          base.Awake();
44          // 前回選んだスロットを PlayerPrefs から読む（なければ 0）
45          CurrentSlot = PlayerPrefs.GetInt(LAST_SLOT_KEY, 0);
46          Load();
47      }
48
49      // ===== スロット =====
50      // スロット番号から保存先を作る（save_0.json, save_1.json …）
51      private static string GetSavePath(int slot)
52      {
53          return Path.Combine(Application.persistentDataPath, $"save_{slot}.json");
54      }
55
56      public bool HasSaveData(int slot)
57      {
58          return File.Exists(GetSavePath(slot));
59      }
```

SaveDataManager.cs（スロット版）

C# Assets/Scripts/SaveDataManager.cs

```
61      // スロットを選んで、そのスロットのデータを読み込む
62      public void SelectSlot(int slot)
63      {
64          CurrentSlot = slot;
65          PlayerPrefs.SetInt(LAST_SLOT_KEY, slot);
66          PlayerPrefs.Save();
67          Load();
68      }
69
70      // スロットのデータを消す
71      public void DeleteSlot(int slot)
72      {
73          if (HasSaveData(slot)) File.Delete(GetSavePath(slot));
74          if (slot == CurrentSlot) ResetData();
75      }
```

SaveDataManager.cs（スロット版）

C# Assets/Scripts/SaveDataManager.cs

```
77      // データを空（はじめから）にする
78      private void ResetData()
79      {
80          isClears = new bool[SceneData.INGAME_COUNT];
81          bestTimes.Clear();
82      }
83
84      // ===== クリア =====
85      public bool IsCleared(int index)
86      {
87          return isClears[index];
88      }
89
90      public void SetClear(int index)
91      {
92          isClears[index] = true;
93          Save();
94      }
```

SaveDataManager.cs（スロット版）

C# Assets/Scripts/SaveDataManager.cs

```
96      // ===== ベストタイム =====
97      /// <summary>
98      /// 引数の値がベストタイムなら保存を更新する
99      /// </summary>
100     /// <param name="sceneName">シーン名</param>
101     /// <param name="time">現在記録</param>
102     /// <returns>更新したら true</returns>
103     public bool TryUpdateBestTime(string sceneName, float time)
104     {
105         bool isGet = bestTimes.TryGetValue(sceneName, out float best);
106         if (isGet && best <= time)
107         {
108             return false;
109         }
110
111         bestTimes[sceneName] = time;
112         Save();
113         return true;
114     }
```

SaveDataManager.cs（スロット版）

C# Assets/Scripts/SaveDataManager.cs

```
116     public bool HasBestTime(string sceneName)
117     {
118         return bestTimes.ContainsKey(sceneName);
119     }
120
121     public float GetBestTime(string sceneName)
122     {
123         return bestTimes[sceneName];
124     }
```

SaveDataManager.cs（スロット版）

C# Assets/Scripts/SaveDataManager.cs

```
126     // ===== セーブ・ロード =====
127     public void Save()
128     {
129         SaveData data = new SaveData();
130         data.isClears = isClears;
131 
132         // Dictionary を List に詰め替える
133         foreach (KeyValuePair<string, float> pair in bestTimes)
134         {
135             BestTimeRecord record = new BestTimeRecord();
136             record.sceneName = pair.Key;
137             record.time = pair.Value;
138             data.bestTimes.Add(record);
139         }
140 
141         string json = JsonUtility.ToJson(data, true);
142         File.WriteAllText(GetSavePath(CurrentSlot), json);
143     }
```

SaveDataManager.cs（スロット版）

C# Assets/Scripts/SaveDataManager.cs

```
145      public void Load()
146      {
147          // まず空にする（前のスロットのデータを残さない）
148          ResetData();
149
150          string path = GetSavePath(CurrentSlot);
151          if (!File.Exists(path)) return;
152
153          string json = File.ReadAllText(path);
154          SaveData data = JsonUtility.FromJson<SaveData>(json);
155
156          isClears = data.isClears;
157
158          // List を Dictionary に戻す
159          foreach (BestTimeRecord record in data.bestTimes)
160          {
161              bestTimes[record.sceneName] = record.time;
162          }
163      }
164  }
```

スロットごとに保存先を変える

C# Assets/Scripts/SaveDataManager.cs

```
50      // スロット番号から保存先を作る（save_0.json, save_1.json …）
51      private static string GetSavePath(int slot)
52      {
53          return Path.Combine(Application.persistentDataPath, $"save_{slot}.json");
54      }
55
56      public bool HasSaveData(int slot)
57      {
58          return File.Exists(GetSavePath(slot));
59      }
```

- **$"save_{slot}.json"**：文字列の中に変数を埋め込む
- **HasSaveData**：そのスロットのファイルがあるか

起動したら、前回のスロットを読み込む

C# Assets/Scripts/SaveDataManager.cs

```
41      protected override void Awake()
42      {
43          base.Awake();
44          // 前回選んだスロットを PlayerPrefs から読む（なければ 0）
45          CurrentSlot = PlayerPrefs.GetInt(LAST_SLOT_KEY, 0);
46          Load();
47      }
```

ポイント

- **PlayerPrefs** から前回のスロット番号
- なければ 0（SLOT 1）
- そのスロットを Load()

SelectSlot — 選び直して読み込み直す

C# Assets/Scripts/SaveDataManager.cs

```
61      // スロットを選んで、そのスロットのデータを読み込む
62      public void SelectSlot(int slot)
63      {
64          CurrentSlot = slot;
65          PlayerPrefs.SetInt(LAST_SLOT_KEY, slot);
66          PlayerPrefs.Save();
67          Load();
68      }
```

ポイント

- 番号を変えて **PlayerPrefs にも保存**（次の起動用）
- そのスロットのデータを **Load()**

ResetData — 前のスロットのデータを残さない

C# Assets/Scripts/SaveDataManager.cs

```
77      // データを空（はじめから）にする
78      private void ResetData()
79      {
80          isClears = new bool[SceneData.INGAME_COUNT];
81          bestTimes.Clear();
82      }
```

ポイント

- **Load の最初**と **DeleteSlot** で呼ぶ
- クリア状況を**全部 false** に
- ベストタイムを**空**に

SaveSlotButton.cs（スロット選択ボタン）

C# Assets/Scripts/SaveSlotButton.cs

```
using TMPro;
using UnityEngine;

// スロット選択画面のボタン（ボタン1つに1つ付ける）
public class SaveSlotButton : MonoBehaviour
{
    [SerializeField] private int slot;  // 0, 1, 2
    [SerializeField] private TextMeshProUGUI label;

    private void Start()
    {
        RefreshLabel();
    }
```

SaveSlotButton.cs（スロット選択ボタン）

C# Assets/Scripts/SaveSlotButton.cs

```
    // Button の OnClick に登録する
    public void OnClickSelect()
    {
        SaveDataManager.Instance.SelectSlot(slot);
        RefreshLabel();
        // ここからステージ選択やゲーム開始へ進む
    }

    private void RefreshLabel()
    {
        bool hasData = SaveDataManager.Instance.HasSaveData(slot);
        string state = hasData ? "つづきから" : "はじめから";
        string mark = SaveDataManager.Instance.CurrentSlot == slot ? "▶ " : "";
        label.text = $"{mark}SLOT {slot + 1}  {state}";
    }
}
```

画面のイメージ

タイトルにボタンを3つ置く

▶ SLOT 1　つづきから
save_0.json がある・今選んでいる

SLOT 2　はじめから
save_1.json がない

SLOT 3　つづきから
save_2.json がある

POINT

各ボタンの **OnClick** に SaveSlotButton.**OnClickSelect** を登録する。選んだあとステージ選択やゲーム開始へ進めるのは、各自のタイトル画面に合わせて足す。

宿題・発展

セーブするものを増やしてみよう

① 音量を PlayerPrefs に

AudioManager に音量を足し、**PlayerPrefs** に保存。
起動時に読み込んで反映する。

② ベストタイムを表示

ステージ選択やリザルトに、**HasBestTime / GetBestTime** で
ベストタイムを出す。

③ スロットに情報を出す

ボタンに「**クリア数**」や「**最後に遊んだ日時**」を出す。
SaveData に何を足す？

④ データを消すボタン

DeleteSlot を使って、スロットを消すボタンを作る。
消す前に確認を出すとなおよい。